SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	**CSA1205**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING**
Weightage:	**45%**	**Total Marks:**	**25**
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	KAVALI BALAJI	**Reg.No.:**	192525402
Department:	AIML	**Date:**	28-07-2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I KAVALI BALAJI (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Naveen Kumar Reddy

MARKS ALLOCATION

	Problem Understanding (20%)	**Method Selection (20%)**	**Solution Process (25%)**	**Accuracy of Calculation (20%)**	**Interpretation of Results (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1: Arithmetic Operations on Signed 8-bit Integers Using 2's Complement

The processor represents signed integers using 8-bit 2's complement notation. In this system:

- Positive numbers are represented in normal binary.
- Negative numbers are represented by taking the 2's complement of the positive value (invert all bits and add 1).
- Range of 8-bit signed integers: **−128 to +127**.

Given:

- +45 = 00101101
- −23 = 11101001 (2's complement of 23)

1. Addition (+45 + (−23))

00101101

+ 11101001

00010110

Result = 00010110 = +22

- Overflow: No (operands have different signs).

2. Subtraction (+45 − (−23))

Convert subtraction into addition:

+45 + (+23)

00101101

+ 00010111

01000100

Result = 01000100 = +68

- Overflow: No (result is within −128 to +127).

Overflow Detection :-

- Overflow occurs when two numbers with the same sign produce a result with the opposite sign.
- In both operations, no overflow occurs.

Conclusion :- The processor represents signed integers using 8-bit 2's complement notation. The values +45 (00101101) and −23 (11101001) are correctly converted before arithmetic operations. During addition, the processor obtains 00010110 (+22), and during subtraction it converts subtraction into addition, producing 01000100 (+68). In both cases, the operands and results remain within the valid 8-bit signed range (−128 to +127), so no overflow occurs.

Q2: Carry Look-Ahead Adder (CLA)

A Carry Look-Ahead Adder (CLA) is a high-speed adder used in modern processors to reduce the delay caused by carry propagation. Unlike a Ripple Carry Adder (RCA), which waits for each carry to pass through every stage, the CLA computes all carry bits simultaneously using Generate (G) and Propagate (P) signals.

Given:

- A = 1010
- B = 0101
- Carry-in (C_0) = 0

Step 1: Generate and Propagate Signals

For each bit:

- Generate (G) = $A \cdot B$
- Propagate (P) = $A \oplus B$

For the given inputs:

Bit	A	B	G = A·B	P = A⊕B
0	0	1	0	1
1	1	0	0	1
2	0	1	0	1
3	1	0	0	1

Step 2: Carry Computation

The carry outputs are calculated in advance using:

- $C_1 = G_0 + P_0C_0$
- $C_2 = G_1 + P_1G_0 + P_1P_0C_0$
- $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$
- $C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

Since $C_0 = 0$ and all G = 0, the carry outputs are:

- $C_1 = 0$
- $C_2 = 0$
- $C_3 = 0$
- $C_4 = 0$

The sum obtained is:
1010 + 0101 = 1111 (Decimal 15)

Comparison with Ripple Carry Adder

Ripple Carry Adder (RCA)	Carry Look-Ahead Adder (CLA)
Carry propagates sequentially	Carries are generated simultaneously
Higher propagation delay	Very low propagation delay
Slower operation	Faster operation
Simple hardware	Slightly complex hardware
Used in low-speed circuits	Used in high-speed processors

Time Delay Comparison

- Ripple Carry Adder: Delay increases linearly with the number of bits (O(n)).
- Carry Look-Ahead Adder: Delay is much smaller because carries are computed in parallel (approximately O(log n)).

Advantages of CLA

- Performs fast binary addition.
- Computes carry bits in parallel.
- Reduces propagation delay.
- Improves CPU and ALU performance.
- Suitable for high-speed computing systems.

Conclusion

A 4-bit Carry Look-Ahead Adder (CLA) improves processor performance by generating Generate (G) and Propagate (P) signals and calculating all carry bits simultaneously. Compared with the Ripple Carry Adder, the CLA has much lower delay, making it ideal for modern high-speed processors and digital systems.

Q3: Booth's Multiplication Algorithm

Booth's Algorithm is an efficient method for multiplying signed binary numbers using 2's complement representation. It reduces the number of addition and subtraction operations, making multiplication faster.

Algorithm Steps

1. Store the multiplicand (M), multiplier (Q), accumulator (A), and Booth bit (Q_{-1}).
2. Examine the last bit of the multiplier (Q_0) and the Booth bit (Q_{-1}):
    - 01 → A = A + M
    - 10 → A = A − M
    - 00 or 11 → No operation
3. Perform an arithmetic right shift on A, Q, and Q_{-1}.
4. Repeat the above steps for the number of bits in the multiplier.
5. The final result is obtained by combining the contents of A and Q.

Role of Registers

- Accumulator (A): Stores intermediate partial products.
- Multiplier (Q): Contains the multiplier and is shifted during each step.
- Booth Bit (Q_{-1}): Stores the previous least significant bit of the multiplier to determine whether addition, subtraction, or no operation is required.

Arithmetic Right Shift

After each operation, an arithmetic right shift is performed on A, Q, and Q_{-1}, preserving the sign bit. This ensures correct multiplication of signed numbers.

Advantages of Booth's Algorithm

- Efficiently handles positive and negative numbers.
- Reduces the number of addition and subtraction operations.
- Improves multiplication speed.
- Widely used in processors, ALUs, and digital signal processors (DSPs).

Example: Booth's Algorithm

Given:

- Multiplicand (M) = −6
- Multiplier (Q) = +5

Using 4-bit 2's complement:

- M = −6 = 1010
- Q = +5 = 0101
- Accumulator (A) = 0000
- Booth bit (Q_{-1}) = 0

Step	Q_0Q_{-1}	Operation	A	Q	Q_{-1}
Initial	–	Initial values	0000	0101	0
1	10	A = A − M	0110	0101	0
Shift	–	Arithmetic Right Shift	0011	0010	1
2	01	A = A + M	1101	0010	1
Shift	–	Arithmetic Right Shift	1110	1001	0
3	10	A = A − M	0100	1001	0
Shift	–	Arithmetic Right Shift	0010	0100	1
4	01	A = A + M	1100	0100	1
Shift	–	Arithmetic Right Shift	1110	0010	0

Final Result

- Product (AQ) = 11100010_2
- Decimal Result = −30

Answer: −6 × +5 = −30

Conclusion

Booth's Algorithm efficiently multiplies −6 and +5 using 2's complement representation and arithmetic right shifts. The final product is −30, and the algorithm is faster than traditional multiplication because it minimizes unnecessary addition and subtraction operation.

Q4: Restoring vs Non-Restoring Division Division

A processor performs binary division using Restoring Division and Non-Restoring Division algorithms. These algorithms divide binary numbers by repeatedly shifting and subtracting the divisor.

1. Restoring Division

Steps

1. Initialize Accumulator (A) = 0000 and load the dividend into Quotient (Q).
2. Left shift A and Q.
3. Subtract the divisor from A.
4. If A becomes negative, restore it by adding the divisor back and set the quotient bit to 0.
5. If A is positive, keep the result and set the quotient bit to 1.
6. Repeat the process for all bits until the quotient is obtained.

Intermediate Result:

- Quotient = 0100_2
- Remainder = 0001_2

2. Non-Restoring Division

Steps

1. Initialize A = 0000 and load the dividend into Q.
2. Left shift A and Q.
3. If A is positive, subtract the divisor.

4. If A is negative, add the divisor instead of restoring.
5. Set the quotient bit based on the sign of A.
6. Repeat for all bits.
7. If the final remainder is negative, add the divisor once to obtain the correct remainder.

Intermediate Result:

- Quotient = 0100_2
- Remainder = 0001_2

Comparison

Restoring Division	Non-Restoring Division
Restores remainder after every negative result	No restoration after each step
More arithmetic operations	Fewer arithmetic operations
More execution steps	Fewer execution steps
Slower	Faster
Higher hardware complexity	Lower hardware complexity

Why Non-Restoring Division is Preferred

- Eliminates unnecessary restoration operations.
- Requires fewer addition and subtraction operations.
- Reduces execution time.
- Improves processor speed and ALU performance.
- Widely used in modern high-speed processors.

Example

- Dividend = 13 = 1101_2
- Divisor = 3 = 0011_2

Final Result:

- Quotient = 4 = 0100_2
- Remainder = 1 = 0001_2

Conclusion

Both Restoring and Non-Restoring Division correctly divide 13 (1101_2) by 3 (0011_2), producing Quotient = 0100_2 (4) and Remainder = 0001_2 (1). However, Non-Restoring Division is preferred because it performs fewer operations, executes faster, and provides better efficiency in modern computer systems.

Q5: IEEE 754 Floating Point Representation

The IEEE 754 standard is used to represent real (floating-point) numbers in computers. It consists of three fields: Sign bit, Exponent, and Mantissa (Fraction).

Example

Given decimal number:
-13.25

Binary form:
$13.25 = 1101.01_2$
Normalized form:
1.10101×2^3

1. Single Precision (32-bit)

- Sign (1 bit) = 1 (Negative number)
- Exponent (8 bits) = $3 + 127 = 130 = 10000010_2$
- Mantissa (23 bits) = 10101000000000000000000

IEEE 754 (32-bit):
1 | 10000010 | 10101000000000000000000

2. Double Precision (64-bit)

- Sign (1 bit) = 1
- Exponent (11 bits) = $3 + 1023 = 1026 = 10000000010_2$
- Mantissa (52 bits) = 1010100000000000000000000000000000000000000000000000

IEEE 754 (64-bit):
1 | 10000000010 | 1010100000000000000000000000000000000000000000000000

Floating-Point Addition

The processor performs floating-point addition using the following steps:

1. Compare the exponents of both numbers.
2. Align the mantissas by shifting the smaller exponent.
3. Add or subtract the mantissas.
4. Normalize the result.
5. Round the result if required.
6. Store the final value in IEEE 754 format.

Issues in Floating-Point Addition

- Normalization: Ensures the result is in the form $1.xxxxx \times 2^n$.
- Rounding: Extra bits are rounded to fit the available mantissa size.
- Precision Loss: Small errors may occur because only a limited number of bits are available.

Comparison

Single Precision	Double Precision
32 bits	64 bits
23-bit mantissa	52-bit mantissa
Lower precision	Higher precision

Single Precision	Double Precision
Less memory	More memory

Conclusion

IEEE 754 represents −13.25 using Sign, Exponent, and Mantissa fields. Single precision (32-bit) provides less accuracy, while double precision (64-bit) offers higher precision. During floating-point addition, the processor aligns exponents, adds mantissas, normalizes the result, and applies rounding. Double precision is preferred for scientific applications because it provides greater accuracy and minimizes precision loss.

1. Align exponents
2. Add mantissas
3. Normalize result
4. Round if needed

Issues:

- Precision loss
- Rounding errors
- Normalization shifts

Thus, IEEE 754 ensures standardized representation but introduces minor precision limitations.